# Метод прогонки

```
import numpy as np

slau = np.matrix(
    [
        [0, 15, 8],
        [2, -15, 4],
        [4, 11, 5],
        [-3, 16, -7],
        [3, 8, 0]
    ]
)

d = [92, -84, -77, 15, -11]

n = 5
P, Q = np.zeros(n + 1, dtype=float), np.zeros(n + 1, dtype=float)

P[0], Q[0] = 0, 0

for i in range(0, n):
    P[i + 1] = -slau[i, 2] / (slau[i, 1] + slau[i, 0] * P[i])
    Q[i + 1] = (d[i] - slau[i, 0] * Q[i]) / (slau[i, 1] + slau[i, 0] * P[i])
```

```
from IPython.display import display, Math

x = np.zeros(n, dtype=float)
for i in range(n, 0, -1):
    Pi = f"{P[i]:.3f}"
    Qi = f"{Q[i]:.3f}"
    if P[i] == 0:
        x[i - 1] = Q[i]
        display(Math(fr"x_{i} = {Qi}"))
        continue
    x[i - 1] = P[i] * x[i] + Q[i]
    xi = f"{x[i - 1]:.3f}"
    display(Math(fr"x_{i} = {xi}"))
```

$x_5 = -1.000$

$x_4 = -1.000$

$x_3 = -8.000$

$x_2 = 4.000$

$x_1 = 4.000$

```
check = np.matrix(
    [
        [15, 8, 0, 0, 0],
```

```
        [2, -15, 4, 0, 0],
        [0, 4, 11, 5, 0],
        [0, 0, -3, 16, -7],
        [0, 0, 0, 3, 8]
    ]
)

print(np.linalg.solve(check, d)[::-1])
```

```
[-1. -1. -8.  4.  4.]
```